

Explorations in Distributed Filesystems

Kelly Xu, Alex Hu, Shreya Chaudhary

1 Introduction

The focus of our project was to build a complete distributed filesystem. With new technologies, we aimed to quantify the performance gain that would be obtained from the integration of different methods. The high level problem we address is: what performance gains and system optimizations are seen with using distributed filesystem frameworks and advanced communication methods, compared to more traditional designs?

To answer this, we present several distributed filesystems implemented with varying tools for comparison. Foremost, we have a custom in-memory distributed filesystem demonstrating a basic client-server system. This model serves as our comparison to examine the performance impact of frameworks and communication methods. Next, we implement a distributed filesystem built on the Filesystem in User Space (FUSE) framework. Finally, we prototype a system using Remote Direct Memory Access (RDMA) for increased performance. These implementations allow us to compare and contrast the performance and use cases of different technologies.

2 Use Cases

We present two example use cases for our system. These are instances where a high-performant distributed filesystem would have the largest impact.

1. **Scenario #1** A distributed file system for a lab server carrying large data files. There are a few large writes and many reads. These reads involve pulling a lot of data from the distributed storage, and benefits from RDMA bypassing the operating system.
2. **Scenario #2** A distributed file system for video / streaming data where there are many small writes and many reads. The high throughput required of streaming data would benefit from performance increases.

3 Code Overview

Our source code is split into two folders: `src` and `src_backup`. The `src` folder contains our FUSE-based file system and our `src_backup` contains our custom-written filesystem.

3.1 FUSE-Based Filesystem

We implement a distributed filesystem with FUSE and gRPC. Clients run our client to mount a FUSE filesystem on their machine. Their filesystem contains a strongly consistent replica of the remote filesystem. This is achieved by each FUSE callback sending an RPC to the server and the server handling RPCs from each client consistently with a global lock. The filesystem supports all functionality, reads, writes, links, symlinks, renames, deletes, syncing to disk, getting stats about the filesystem, leading to 23 callbacks implemented. We disable Direct IO which means the kernel page cache is used for file data; this allows the OS to cache read and write data locally, but also introduces cache coherence issues where updates from other clients or the server may not be immediately reflected. We rely on FUSE's default behavior where cached data is invalidated according to kernel heuristics or upon close.

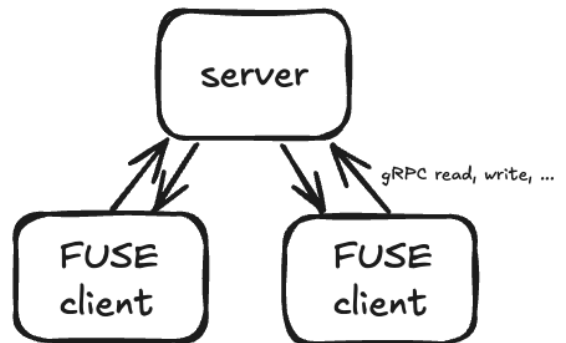


Figure 1: Overview of our FUSE filesystem.

3.2 Handwritten Filesystem

As shown in figure 2, our handwritten filesystem abstraction contains three main components: the Coordinator, the Server, and the Client. On a request, a Client sends a message to the Coordinator. The Coordinator replies with which server the Client should communicate to which is selected by hashing the file name. Finally, the Client communicates with its assigned Server to perform operations such as write and read.

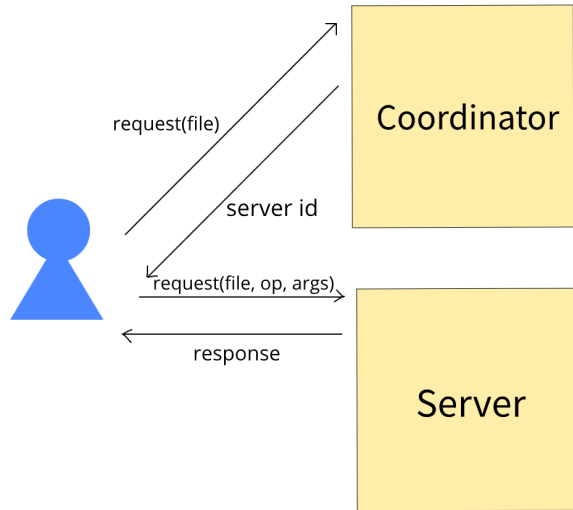


Figure 2: Overview of the requests between our filesystem.

The coordinator is completely lock-free and simply hashes the file to one of the servers to be used. The servers support reading and writing files in arbitrary paths. There is currently a write lock but not a read lock, so the readers could get slightly outdated information. In the context of scenario #2, this is okay since viewers can tolerate the most recent video being slightly outdated. However, in the context of scenario #1, it might not be tolerable to outdated data, so we ought to add a lock for both reads and writes.

3.3 RDMA filesystem

We prototype an extension of our FUSE filesystem with RDMA. The bottleneck in FUSE is the transferring of file read and write data over TCP, which can be done more efficiently with RDMA. We have a control-plane gRPC service that answers only metadata-centric RPCs—open, getattr, rename, unlink, allocate_extent, commit, and so on—while maintaining per-inode version numbers and lightweight CAS-based locks instead of a single global mutex. In parallel, the server pre-registers one or more gigabyte-sized huge-page regions with `IBV_ACCESS_LOCAL_WRITE | REMOTE_READ | REMOTE_WRITE | REMOTE_ATOMIC`. These regions are carved into fixed-size extents (for example 4 MiB each),

and each extent’s (offset, length, raddr, rkey, version) entry is stored in the inode’s extent map that the control-plane returns to clients at open time or on demand. These regions are periodically flushed to memory by a background thread.

During a read, the FUSE callback consults the extent map it obtained at open. If the version cached in userspace still matches the inode’s current version, the client issues one-sided `ibv_post_send()` work requests of type `RDMA_READ`, copying data directly from the extent’s remote address into the page cache buffer that FUSE provided. The server CPU is not involved; the NIC performs DMA into the client. If the client notices a version mismatch—detected by rereading the 64-bit version word packed into the inode header after the RDMA READ completes—it simply asks the control-plane for an updated extent map and retries.

For a write, the client first sends an `AllocateExtents` RPC telling the server how many bytes it will write. The server finds free extents, marks them “reserved,” and replies with the (raddr, rkey) pairs plus an exclusive lock token (implemented as a successful `CMP_AND_SWAP` on a 64-bit lock word). The client then performs one-sided `RDMA_WRITE` operations into those extents. After all RDMA WRITES complete—and, optionally, after computing a CRC or SHA-256 in userspace for integrity—the client issues a `Commit` RPC that carries the list of written ranges, the expected lock token, and the new version number. The server validates the checksum, updates the inode’s extent map atomically, bumps the version, flushes or invalidates the memory region if you enable MR-invalidation, and finally releases the lock. Crash-consistency follows the classical write-ahead-metadata rule: data reach stable storage (the registered memory) before metadata becomes visible, so any crash before `Commit` leaves unreferenced “orphan” extents that a recovery scan can reclaim safely.

4 Results

4.1 FUSE Results

We tested our FUSE filesystem with varying read/write ratios, shown in Figures 3 and 4 with the FIO tool, with block size of 4k, total memory IO of 10 MB, a random mixed read/write workload with varying write percentage, 1 job, and the Linux Asynchronous IO engine that’s supposed to enable higher throughput and lower latency for workloads that can benefit from parallelism. Note that the global lock causes contention which decreases bandwidth and increases latency. Performance is roughly an order of magnitude slower than if we had run this benchmark on our local filesystem. Sending large amounts of data over TCP is a bottleneck, as well as the global lock.

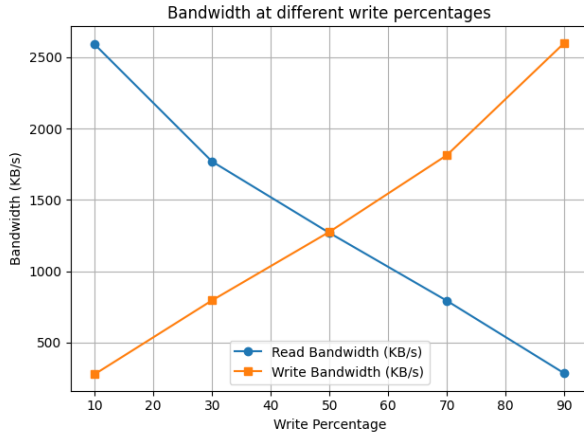


Figure 3: Results of our FUSE filesystem.

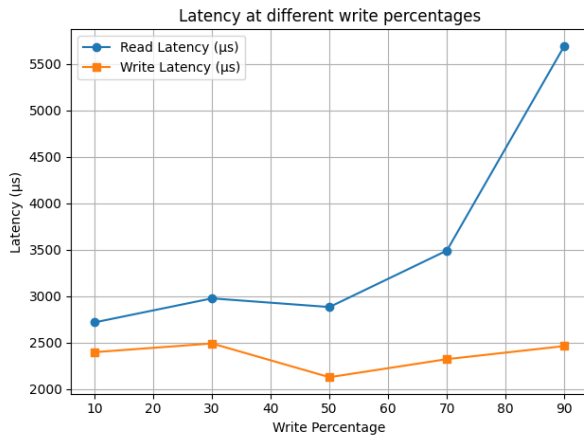


Figure 4: Results of our FUSE filesystem.

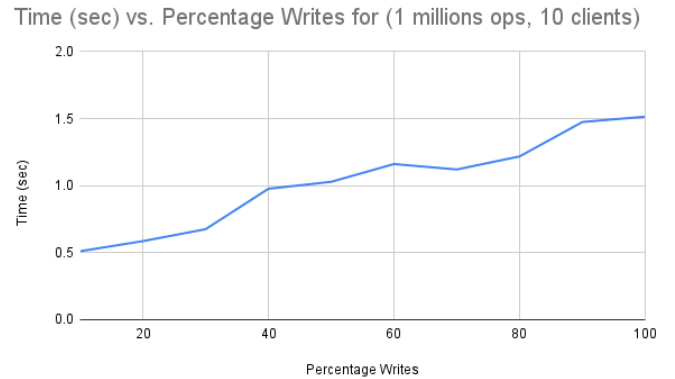


Figure 5: Results of our handwritten filesystem (time across ratio of writes to reads).

4.2 Handwritten Filesystem Results

We tested our handwritten filesystem with varying read/write ratios, as demonstrated in Figure 5.

Each write was relatively small, only a few bytes. The tests were run with 1 million operations across 10 clients with different percentages of writes.

5 Code Availability

Our code is open-sourced and available at <https://github.com/mit-class-projects/fs>